DM&V

Practical Experiments

Experiments 1 to 6

Index

Exp. No.	Title of Experiment	Page
1	Data Loading, Storage and File Formats	2
2	Interacting with Web APIs	7
3	Data Cleaning and Preparation	11
4	Data Wrangling	15
5	Data Visualization Using Matplotlib	19
6	Data Aggregation	23

EXPERIMENT 1

Data Loading, Storage and File Formats

1. Title

Analyzing Sales Data from Multiple File Formats (CSV, Excel and JSON)

2. Objectives

1. To load sales data from different file formats such as CSV, Excel and JSON.
2. To explore the structure and contents of the datasets.
3. To identify missing values, duplicate records and data inconsistencies.
4. To clean and preprocess the sales data.
5. To convert datasets into a common/unified format.
6. To merge and transform data for analysis.
7. To calculate important sales metrics such as total sales and average order value.
8. To analyze sales according to product categories.
9. To create suitable visualizations for understanding sales performance.

3. Problem Statement

Sales data is often stored in different file formats such as CSV, Excel and JSON. These files may contain missing values, duplicate records, inconsistent data types and different representations of the same information.

The objective of this practical is to load sales data from CSV, Excel and JSON files, explore and clean the data, convert it into a common DataFrame, merge the datasets and perform sales analysis. Descriptive statistics, aggregation and visualization techniques are used to obtain meaningful insights about sales, products and customers.

4. Theory

4.1 Data Loading

Data loading is the process of importing data from external files into a programming environment. Python's Pandas library provides functions for loading data from various formats.

Examples:

- pd.read_csv() → loads CSV files
- pd.read_excel() → loads Excel files
- pd.read_json() → loads JSON files

4.2 CSV

CSV stands for **Comma-Separated Values**. It stores tabular data in plain text where values are separated by commas.

Example:

```
Order_ID,Product,Category,Quantity,Price
101,Laptop,Electronics,2,50000
102,Chair,Furniture,3,5000
```

4.3 Excel

Excel files generally have .xlsx or .xls extensions. They are commonly used for storing structured tabular data.

Python can read Excel files using Pandas and the OpenPyXL library.

4.4 JSON

JSON stands for **JavaScript Object Notation**. It stores data in a structured key-value format and is widely used for exchanging data between applications.

Example:

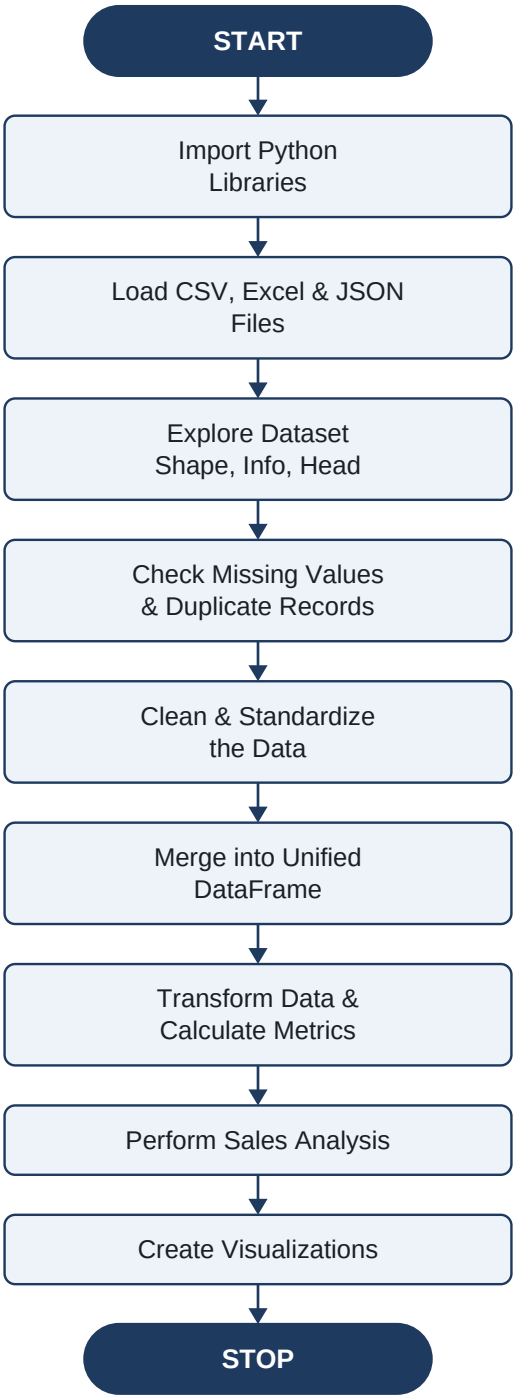
```
[
    {
        "Order_ID": 105,
        "Product": "Mobile",
        "Category": "Electronics",
        "Quantity": 2,
        "Price": 20000
    }
]
```

5. Algorithm

1. Start.
2. Import the required Python libraries.
3. Create/load sales datasets in CSV, Excel and JSON formats.
4. Read each file using the appropriate Pandas function.
5. Display the first few records of each dataset.
6. Check the structure, data types and dimensions.
7. Check for missing values and duplicate records.
8. Remove duplicate records.
9. Handle missing values.
10. Standardize column names and data types.
11. Combine the datasets into a single DataFrame.
12. Calculate Total_Sales = Quantity × Price.
13. Calculate total sales and average order value.
14. Group the data by product category.

15. Create bar chart, pie chart and box plot.
16. Analyze the results.
17. Stop.

6. Flowchart



7. Test Cases

Test Case	Input / Operation	Expected Result
TC01	Load CSV file	CSV data loaded successfully
TC02	Load Excel file	Excel data loaded successfully
TC03	Load JSON file	JSON data loaded successfully
TC04	Check dataset shape	Number of rows and columns displayed
TC05	Check missing values	Missing-value count displayed
TC06	Check duplicates	Duplicate records identified
TC07	Remove duplicates	Duplicate records removed
TC08	Calculate Total Sales	Quantity × Price calculated
TC09	Category aggregation	Category-wise sales displayed
TC10	Calculate average order value	Average sales per order displayed
TC11	Generate bar plot	Category-wise sales chart displayed
TC12	Generate pie chart	Category distribution displayed
TC13	Generate box plot	Sales distribution displayed

8. Test Data Set

The following sample dataset is used:

Order_ID	Product	Category	Quantity	Price
101	Laptop	Electronics	2	50000
102	Mobile	Electronics	3	20000
103	Chair	Furniture	4	5000
104	Table	Furniture	2	8000
105	Laptop	Electronics	1	50000
106	Mobile	Electronics	5	20000
107	Headphones	Electronics	3	3000
108	Chair	Furniture	2	5000
109	Table	Furniture	4	8000
110	Laptop	Electronics	1	50000

The same dataset is stored in:

- sales_data.csv
- sales_data.xlsx
- sales_data.json

9. Conclusion

The practical successfully demonstrated how sales data can be loaded from **CSV, Excel and JSON** file formats using Python and Pandas.

The datasets were explored, cleaned and converted into a common structure. Duplicate records and missing values were handled, and the datasets were merged into a unified DataFrame. Data transformation was performed by deriving the total_sales column.

EXPERIMENT 2

Interacting with Web APIs

1. Title

Analyzing Weather Data Using OpenWeatherMap API

2. Objectives

1. To understand the concept of Web APIs and REST APIs.
2. To register with OpenWeatherMap and obtain an API key.
3. To interact with the OpenWeatherMap API using Python.
4. To retrieve weather information for a specific location.
5. To extract temperature, humidity, wind speed and precipitation data.
6. To clean and preprocess the retrieved weather data.
7. To calculate average, minimum and maximum weather values.
8. To analyze weather patterns over time.
9. To visualize weather attributes using suitable graphs.
10. To study relationships between weather parameters using correlation analysis.
11. To perform time-based aggregation of weather data.

3. Problem Statement

Weather information is available through online Web APIs that provide real-time and historical weather-related information. The objective of this practical is to interact with the **OpenWeatherMap API** using an API key, retrieve weather data for a specified location, extract important weather attributes, clean and preprocess the data, and perform statistical analysis.

4. Theory

4.1 Web API

API stands for **Application Programming Interface**.

A Web API allows one software application to communicate with another application over the internet.

For example, a Python program can send a request to OpenWeatherMap and receive weather information in JSON format.

4.2 REST API

REST stands for **Representational State Transfer**.

REST APIs commonly use HTTP methods such as:

- GET – retrieve data

- POST – send data
- PUT – update data
- DELETE – remove data

For retrieving weather information, the **GET** method is generally used.

4.3 OpenWeatherMap API

OpenWeatherMap provides weather information through APIs. A request generally contains:

- API endpoint
- Location
- API key
- Units such as metric or imperial

The API returns structured data, usually in **JSON format**.

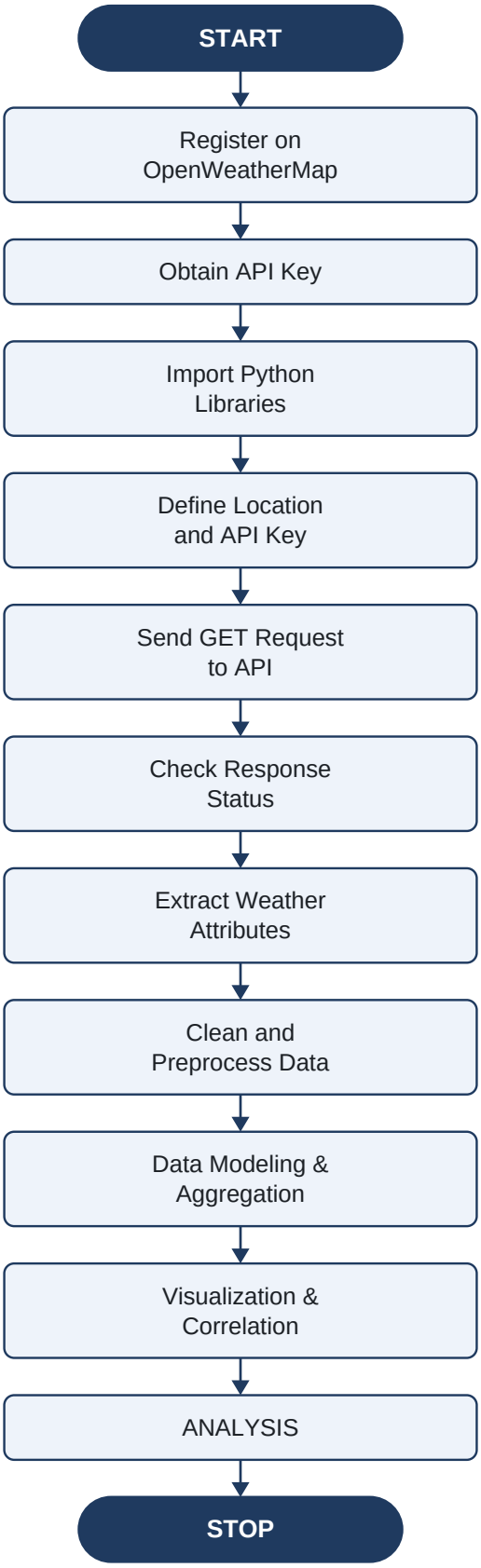
A typical request conceptually looks like:

```
API Endpoint + Location + API Key + Units
```

5. Algorithm

1. Start.
2. Register on the OpenWeatherMap website.
3. Obtain an API key.
4. Import the required Python libraries.
5. Define the location and API key.
6. Construct the API request URL.
7. Send a GET request to the API.
8. Check the HTTP response status.
9. Convert the JSON response into Python data.
10. Extract temperature, humidity, wind speed and precipitation.
11. Store the extracted data in a Pandas DataFrame.
12. Check for missing and invalid values.
13. Clean and preprocess the data.
14. Convert timestamps into date/time format.
15. Calculate average, minimum and maximum values.
16. Perform time-based aggregation.
17. Calculate correlations between weather attributes.
18. Create suitable visualizations.
19. Analyze the results.
20. Stop.

6. Flowchart


START
Register on OpenWeatherMap
Obtain API Key
Import Python Libraries
Define Location and API Key
Send GET Request to API
Check Response Status
Extract Weather Attributes
Clean and Preprocess Data
Data Modeling & Aggregation
Visualization & Correlation
ANALYSIS
STOP

7. Test Cases

Test Case	Input / Operation	Expected Result
TC01	Enter valid API key	API request should be successful
TC02	Enter invalid API key	API should return an authentication error
TC03	Enter valid city	Weather data should be retrieved
TC04	Enter invalid city	Location-not-found error should be returned
TC05	Extract temperature	Temperature should be displayed in °C
TC06	Extract humidity	Humidity should be displayed as percentage
TC07	Extract wind speed	Wind speed should be displayed
TC08	Extract precipitation	Rainfall should be displayed or default handled if unavailable
TC09	Check missing values	Missing values should be identified
TC10	Calculate average temperature	Average temperature should be calculated
TC11	Calculate min/max temperature	Minimum and maximum should be displayed
TC12	Generate line plot	Temperature trend should be displayed
TC13	Generate scatter plot	Temperature-humidity relationship should be displayed
TC14	Generate heatmap	Correlation matrix should be visualized

8. Conclusion

The practical successfully demonstrated how to **interact with a Web API using Python**. The OpenWeatherMap API was used to retrieve weather information for a specified location.

The retrieved JSON data was converted into a structured DataFrame and cleaned using Pandas. Important weather attributes such as temperature, humidity, wind speed and precipitation were extracted and analyzed.

EXPERIMENT 3

Data Cleaning and Preparation

1. Title

Data Cleaning and Preparation for Analyzing Customer Churn in a Telecommunications Company

2. Objectives

1. To import the Telecom_Customer_Churn.csv dataset.
2. To explore the structure and contents of the dataset.
3. To identify and handle missing values.
4. To remove duplicate records.
5. To identify and correct inconsistent data.
6. To convert variables into appropriate data types.
7. To identify and handle outliers.
8. To perform feature engineering.
9. To normalize or scale numerical features where required.
10. To divide the dataset into training and testing sets.
11. To export the cleaned dataset for further analysis and machine-learning applications.

3. Problem Statement

A telecommunications company wants to understand the factors responsible for customer churn. Customer churn occurs when customers discontinue their services.

The dataset Telecom_Customer_Churn.csv contains customer demographic information, service usage information, account details and churn status. Before applying statistical analysis or machine-learning algorithms, the dataset must be cleaned and prepared.

4. Theory

4.1 Data Cleaning

Data cleaning is the process of identifying and correcting errors, missing values, duplicate records and inconsistent information in a dataset.

It improves data quality and makes the dataset suitable for analysis and machine learning.

4.2 Missing Values

Missing values occur when information is not available for a particular record.

Common techniques for handling missing values are:

- Removing records
- Replacing with mean
- Replacing with median
- Replacing with mode
- Forward/backward filling

4.3 Duplicate Records

Duplicate records are repeated observations in a dataset. They can affect statistical calculations and model performance.

Pandas provides:

```
df.duplicated()
```

and

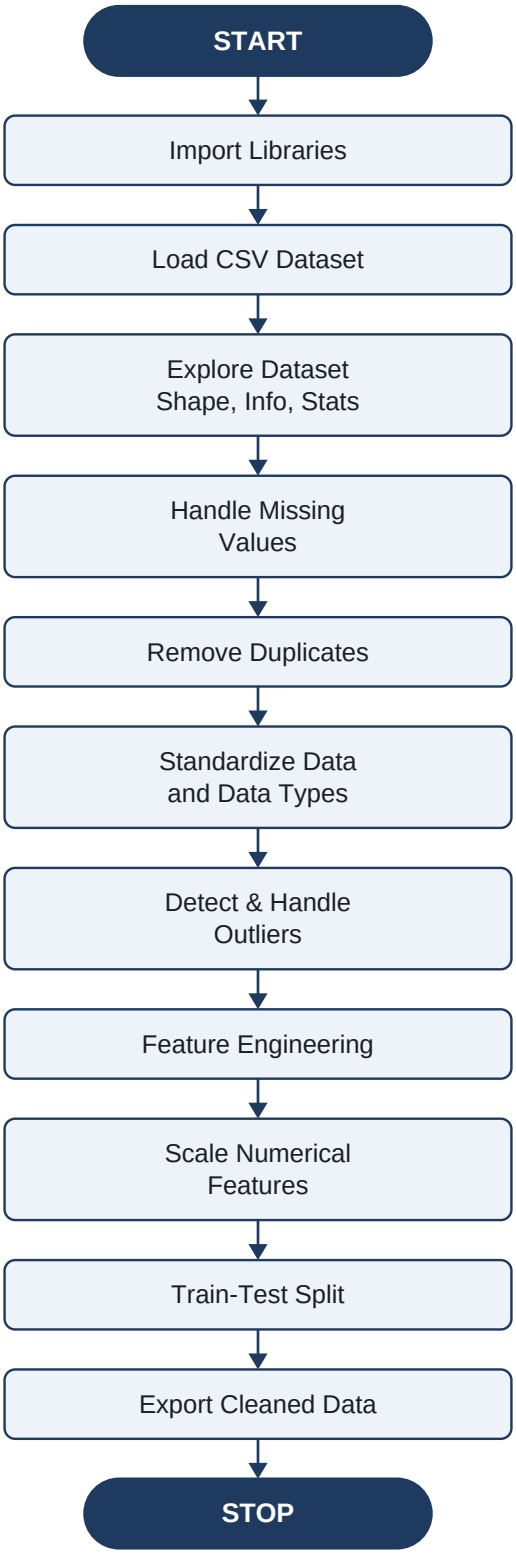
```
df.drop_duplicates()
```

for identifying and removing duplicates.

5. Algorithm

1. Start.
2. Import the required Python libraries.
3. Load Telecom_Customer_Churn.csv.
4. Display the first few records.
5. Check dataset shape, columns and data types.
6. Identify missing values.
7. Handle missing values using suitable methods.
8. Identify and remove duplicate records.
9. Standardize categorical values.
10. Convert columns into appropriate data types.
11. Identify numerical outliers using the IQR method.
12. Handle extreme outliers using suitable limits.
13. Create new features using feature engineering.
14. Separate input features and target variable.
15. Scale numerical features.
16. Split the dataset into training and testing sets.
17. Export the cleaned dataset.
18. Analyze the cleaned dataset.
19. Stop.

6. Flowchart


START
Import Libraries
Load CSV Dataset
Explore Dataset
Shape, Info, Stats
Handle Missing
Values
Remove Duplicates
Standardize Data
and Data Types
Detect & Handle
Outliers
Feature Engineering
Scale Numerical
Features
Train-Test Split
Export Cleaned Data
STOP

7. Test Cases

Test Case	Operation	Expected Result
TC01	Load CSV file	Dataset loaded successfully
TC02	Display first records	First records displayed
TC03	Check dataset shape	Rows and columns displayed
TC04	Check missing values	Missing-value count displayed
TC05	Handle missing values	Missing values replaced appropriately
TC06	Check duplicates	Duplicate count displayed
TC07	Remove duplicates	Duplicate records removed
TC08	Standardize text	Inconsistent formatting corrected
TC09	Convert data types	Columns converted to correct types
TC10	Detect outliers	Outliers identified using IQR
TC11	Handle outliers	Extreme values capped
TC12	Create tenure group	New feature created
TC13	Create spending feature	Average spending calculated
TC14	Encode churn	Churn converted into 0/1
TC15	Train-test split	Data divided into 80:20 ratio
TC16	Scale features	Numerical features standardized
TC17	Export data	Cleaned CSV generated

8. Conclusion

The practical successfully demonstrated the complete process of **data cleaning and preparation for customer churn analysis**.

EXPERIMENT 4

Data Wrangling

1. Title

Data Wrangling on Real Estate Market

2. Objectives

1. To import the RealEstate_Prices.csv dataset.
2. To clean and standardize column names.
3. To identify and handle missing values.
4. To merge additional datasets when relevant information is available.
5. To filter and subset the real estate data based on specific conditions.
6. To encode categorical variables for analysis and modeling.
7. To aggregate data and calculate summary statistics.
8. To identify and handle outliers and extreme values.
9. To prepare a clean and structured dataset for further analysis.

3. Problem Statement

The real estate market contains information about properties, including property characteristics, locations, property types and sale prices. Such datasets may contain missing values, inconsistent column names, categorical variables and extreme property prices.

The objective of this practical is to perform **data wrangling** on RealEstate_Prices.csv. The dataset is cleaned and transformed by standardizing column names, handling missing values, filtering records, encoding categorical variables, aggregating data and identifying outliers. The resulting dataset can be used for further statistical analysis and machine-learning applications.

4. Theory

4.1 Data Wrangling

Data wrangling is the process of collecting, cleaning, transforming, organizing and preparing raw data for analysis.

It includes operations such as:

- Cleaning column names
- Handling missing values
- Merging datasets
- Filtering records
- Encoding categorical variables

- Aggregating data
- Handling outliers

4.2 Column Name Cleaning

Raw datasets may contain column names such as:

```
Sale Price
Property Type
Lot Area (sq.ft.)
```

These can be converted into standardized names:

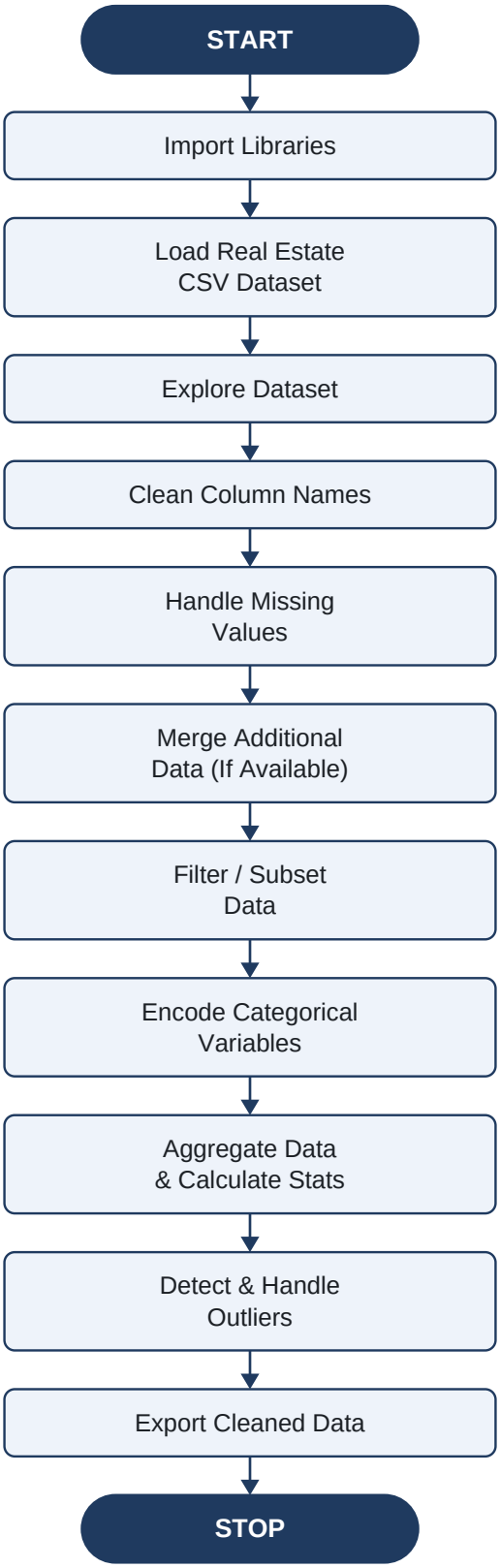
```
sale_price
property_type
lot_area_sq_ft
```

Standardized column names make programming and analysis easier.

5. Algorithm

1. Start.
2. Import required Python libraries.
3. Load RealEstate_Prices.csv.
4. Display the first and last records.
5. Examine dataset shape, columns and data types.
6. Clean and standardize column names.
7. Identify missing values.
8. Handle missing values using appropriate methods.
9. Load an optional additional dataset if available.
10. Merge the datasets using a common key.
11. Filter the dataset based on property type, location or price.
12. Encode categorical variables.
13. Calculate aggregate statistics such as average price.
14. Detect outliers using the IQR method.
15. Handle extreme values.
16. Export the cleaned and transformed dataset.
17. Analyze the results.
18. Stop.

6. Flowchart


START
Import Libraries
Load Real Estate CSV Dataset
Explore Dataset
Clean Column Names
Handle Missing Values
Merge Additional Data (If Available)
Filter / Subset Data
Encode Categorical Variables
Aggregate Data & Calculate Stats
Detect & Handle Outliers
Export Cleaned Data
STOP

7. Test Cases

Test Case	Operation	Expected Result
TC01	Load CSV dataset	Dataset loaded successfully
TC02	Display dataset shape	Rows and columns displayed
TC03	Clean column names	Spaces/special characters removed
TC04	Check missing values	Missing-value count displayed
TC05	Handle missing values	Missing values replaced
TC06	Check duplicate records	Duplicate count displayed
TC07	Remove duplicates	Duplicate records removed
TC08	Standardize text values	Unnecessary spaces removed
TC09	Convert numeric columns	Correct numeric data types obtained
TC10	Merge additional data	Datasets merged using common key
TC11	Filter properties	Required subset obtained
TC12	Encode categorical variables	Categorical variables converted to numerical form
TC13	Aggregate prices	Average/median prices calculated
TC14	Detect outliers	Outliers identified using IQR
TC15	Handle outliers	Extreme values capped
TC16	Calculate derived feature	Price per square foot calculated
TC17	Export dataset	Cleaned CSV generated

8. Conclusion

The practical successfully demonstrated the process of **data wrangling on a real estate dataset**.

The RealEstate_Prices.csv dataset was cleaned by standardizing column names, handling missing values, removing duplicates and correcting data types. Additional datasets could be merged using common attributes such as neighborhood.

EXPERIMENT 5

Data Visualization Using Matplotlib

1. Title

Analyzing Air Quality Index (AQI) Trends in a City Using Matplotlib

2. Objectives

1. To import the City_Air_Quality.csv dataset.
2. To explore the structure and contents of the air quality dataset.
3. To identify relevant variables such as date, PM2.5, PM10, CO and AQI.
4. To visualize AQI trends over time using line plots.
5. To visualize individual pollutant trends using line plots.
6. To compare AQI values across different dates using bar plots.
7. To analyze the distribution of AQI values using box plots.
8. To study relationships between AQI and pollutant levels using scatter plots.
9. To customize graphs using titles, labels, legends and suitable colors.
10. To gain meaningful insights into air quality trends in the city.

3. Problem Statement

Air quality is an important environmental factor that affects human health and the overall quality of life in a city. Air quality datasets contain measurements of pollutants such as PM2.5, PM10 and CO along with the Air Quality Index (AQI).

The objective of this practical is to use the **Matplotlib** library to visualize AQI trends and pollutant levels over time. Different visualization techniques such as line plots, bar plots, box plots and scatter plots are used to identify patterns, trends, distributions and relationships between AQI and different pollutants.

4. Theory

4.1 Data Visualization

Data visualization is the graphical representation of data. It helps users understand trends, patterns, comparisons and relationships within a dataset.

Common visualization techniques include:

- Line plot
- Bar plot
- Scatter plot
- Box plot
- Histogram

- Pie chart

4.2 Matplotlib

Matplotlib is a popular Python library used for creating static and interactive graphs.

It supports different types of plots, including:

```
plt.plot()
plt.bar()
plt.scatter()
plt.boxplot()
plt.hist()
```

4.3 Air Quality Index

The **Air Quality Index (AQI)** is an index used to represent air quality based on the concentration of pollutants.

Pollutants commonly considered in air-quality monitoring include:

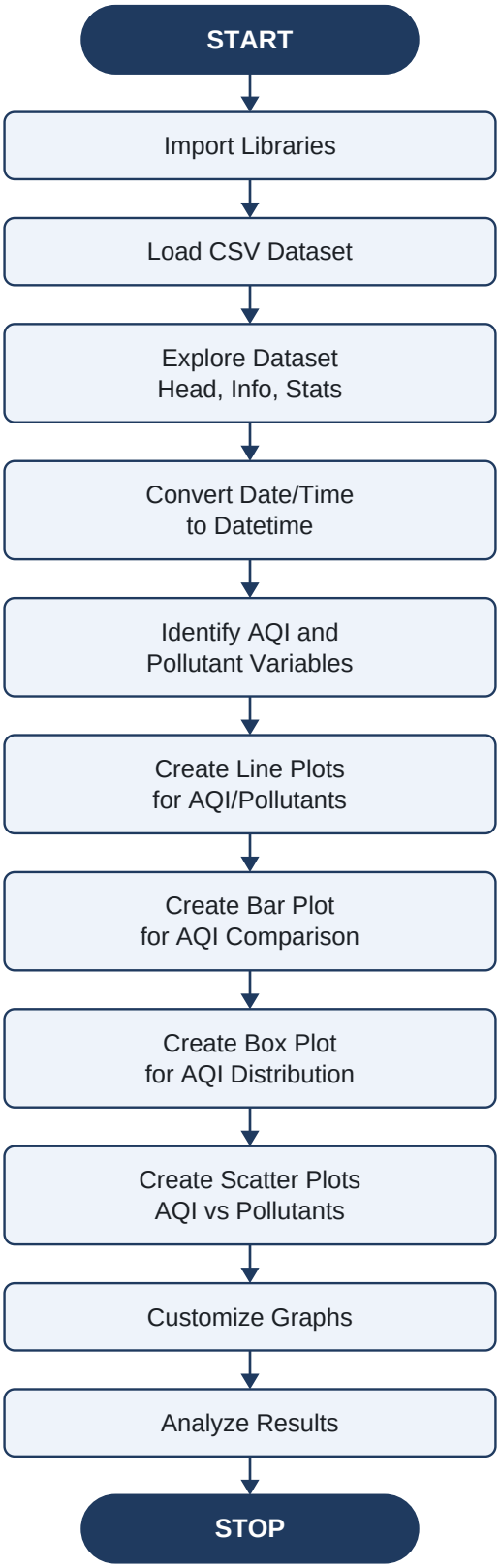
- PM2.5
- PM10
- CO
- NO_2
- SO_2
- O_3

Higher pollutant concentrations can generally correspond to poorer air quality, depending on the AQI calculation standard being used.

5. Algorithm

1. Start.
2. Import Pandas, NumPy and Matplotlib.
3. Load City_Air_Quality.csv.
4. Explore the dataset using head(), info() and describe().
5. Convert the date column into datetime format.
6. Identify AQI and pollutant columns.
7. Create a time-series line plot for AQI.
8. Create separate line plots for PM2.5, PM10 and CO.
9. Create a bar plot to compare AQI values across dates.
10. Create a box plot to analyze AQI distribution.
11. Create scatter plots between AQI and pollutants.
12. Add titles, labels, legends and grid lines.
13. Analyze the graphs and identify trends.
14. Stop.

6. Flowchart


START
Import Libraries
Load CSV Dataset
Explore Dataset
Head, Info, Stats
Convert Date/Time
to Datetime
Identify AQI and
Pollutant Variables
Create Line Plots
for AQI/Pollutants
Create Bar Plot
for AQI Comparison
Create Box Plot
for AQI Distribution
Create Scatter Plots
AQI vs Pollutants
Customize Graphs
Analyze Results
STOP

7. Test Cases

Test Case	Operation	Expected Result
TC01	Import CSV dataset	Dataset loaded successfully
TC02	Display dataset structure	Shape and columns displayed
TC03	Check missing values	Missing values identified
TC04	Convert date/time	Date-time column converted successfully
TC05	Plot overall AQI	AQI trend line displayed
TC06	Plot PM2.5	PM2.5 trend displayed
TC07	Plot PM10	PM10 trend displayed
TC08	Plot CO	CO trend displayed
TC09	Create AQI bar plot	Daily AQI comparison displayed
TC10	Create AQI box plot	AQI distribution displayed
TC11	Create scatter plot	AQI-pollutant relationship displayed
TC12	Calculate correlation	Correlation matrix generated
TC13	Customize graphs	Titles, labels, legends and grids displayed

8. Conclusion

The practical successfully demonstrated **data visualization using Matplotlib** for analyzing Air Quality Index trends.

The City_Air_Quality.csv dataset was imported and explored, and relevant variables such as AQI, PM2.5, PM10 and CO were identified. Line plots were used to study changes in AQI and pollutant levels over time.

EXPERIMENT 6

Data Aggregation

1. Title

Analyzing Sales Performance by Region in a Retail Company Using Data Aggregation

2. Objectives

1. To import the Retail_Sales_Data.csv dataset.
2. To explore the structure and contents of the sales dataset.
3. To identify relevant variables such as region, sales amount and product category.
4. To aggregate sales data by region.
5. To calculate total sales for each region.
6. To visualize regional sales using bar plots and pie charts.
7. To identify the top-performing regions.
8. To aggregate sales by region and product category.
9. To compare sales performance across regions and product categories using grouped or stacked bar plots.

3. Problem Statement

A retail company generates a large number of sales transactions across different geographical regions and product categories. Analyzing individual transactions may not provide a clear understanding of overall business performance.

4. Theory

4.1 Data Aggregation

Data aggregation is the process of combining multiple records into summary information.

For example, instead of analyzing every individual sales transaction, we can calculate:

- Total sales by region
- Average sales by region
- Total quantity sold by product
- Sales by region and product category

Pandas provides the groupby() function for performing aggregation.

4.2 GroupBy Operation

The groupby() function divides data into groups based on one or more columns.

Example:

```
df.groupby("region")["sales_amount"].sum()
```

This calculates the total sales amount for every region.

4.3 Aggregation Functions

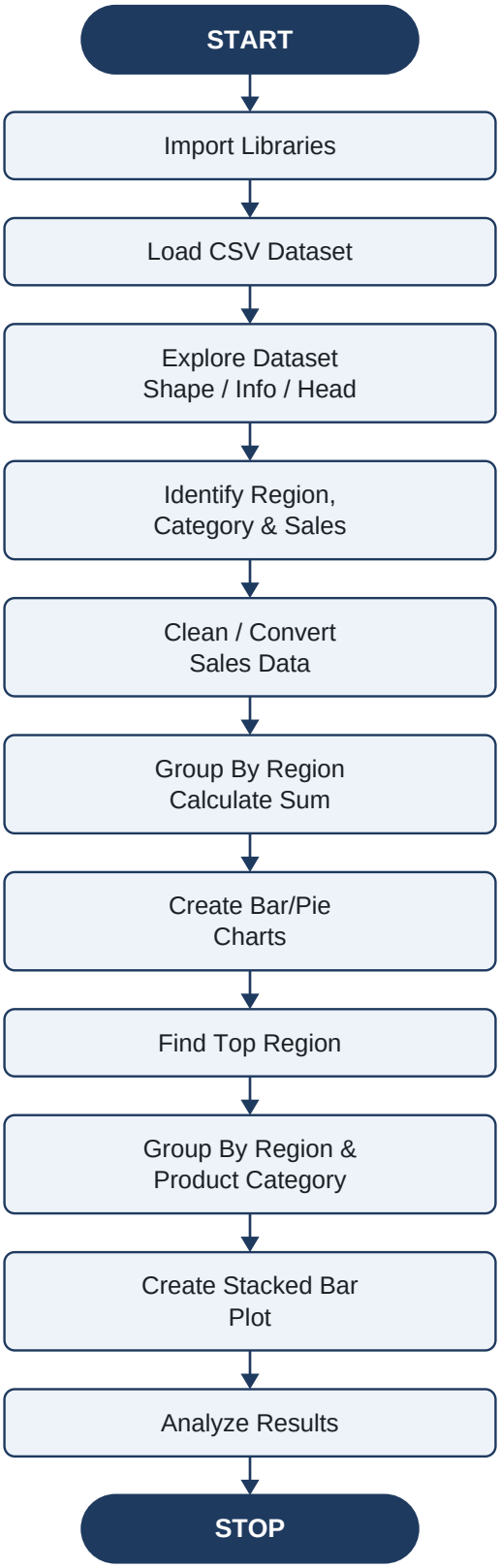
Common aggregation functions include:

Function	Purpose
sum()	Calculates total
mean()	Calculates average
min()	Finds minimum
max()	Finds maximum
count()	Counts records
median()	Finds median

5. Algorithm

1. Start.
2. Import Pandas, NumPy and Matplotlib.
3. Load Retail_Sales_Data.csv.
4. Display the first few records.
5. Examine dataset shape, columns and data types.
6. Check for missing values and duplicates.
7. Identify region, product category and sales amount columns.
8. Convert sales amount into numerical format.
9. Group the dataset by region.
10. Calculate total sales for each region.
11. Sort regions according to total sales.
12. Create a bar plot of regional sales.
13. Create a pie chart showing regional sales distribution.
14. Identify the top-performing region.
15. Group data by region and product category.
16. Calculate total sales for each combination.
17. Create a stacked bar plot.
18. Analyze the results.
19. Stop.

6. Flowchart


START
Import Libraries
Load CSV Dataset
Explore Dataset
Shape / Info / Head
Identify Region,
Category & Sales
Clean / Convert
Sales Data
Group By Region
Calculate Sum
Create Bar/Pie
Charts
Find Top Region
Group By Region &
Product Category
Create Stacked Bar
Plot
Analyze Results
STOP

7. Test Cases

Test Case	Operation	Expected Result
TC01	Import CSV	Dataset loaded successfully
TC02	Display dataset	First records displayed
TC03	Check shape	Number of rows and columns displayed
TC04	Check missing values	Missing values identified
TC05	Remove duplicates	Duplicate records removed
TC06	Convert sales amount	Sales converted to numeric format
TC07	Group by region	Regional sales calculated
TC08	Sort regional sales	Regions ranked by sales
TC09	Create bar plot	Regional sales visualized
TC10	Create pie chart	Sales distribution visualized
TC11	Find top region	Highest-selling region identified
TC12	Group region/category	Combined sales calculated
TC13	Create pivot table	Region-category matrix generated
TC14	Stacked bar plot	Category contributions compared
TC15	Grouped bar plot	Regional category sales compared

8. Conclusion

The practical successfully demonstrated **data aggregation for analyzing sales performance by region**.

The Retail_Sales_Data.csv dataset was imported, explored and prepared for analysis. Sales data was grouped by region to calculate total sales and identify the top-performing region.